MILESTONE 1 · DATA ENGINEER

Penjelasan Semua Tabel Dataset Omnichannel Retail

Apa isi tiap tabel, satu baris mewakili apa, dan untuk apa tabel itu dipakai

19	158.770	4	90
FILE DATASET TERSEDIA	TOTAL BARIS MENTAH	KELUARGA SUMBER	HARI DATA (22 JUN–19 SEP 2026)

Proyek	Omnichannel Retail NL-to-SQL Analytics
Sumber data	Google Drive data lake → data/raw/
Format	17 file JSON + 2 file CSV, dikelompokkan dalam 4 folder
Peruntukan	Dokumen belajar pribadi — tidak dikumpulkan saat submission

STATUS DATA: LENGKAP

Seluruh **19 file sudah tersedia**. Empat file yang sebelumnya belum dibagikan instructor — order_promotions.json, payment_events.json, refund_events.json, dan web_events.json — kini sudah ada dan sudah ikut diproses pipeline. Dokumen ini memakai angka dari run terakhir yang mencakup keempatnya.

Seluruh angka pada dokumen ini diambil langsung dari database pipeline (bronze, silver, gold) dan dari file mentah di data/raw/.

1 · Gambaran besar

Baca bagian ini dulu sebelum masuk ke tabel satu per satu.

Dataset ini mewakili satu perusahaan retail yang menjual lewat empat channel sekaligus (omnichannel): website, aplikasi mobile, marketplace, dan toko fisik. Datanya sengaja dibelah menjadi empat folder karena berasal dari empat sistem yang berbeda di dunia nyata — sistem transaksi, sistem event, sistem gudang, dan sistem marketing.

```
data/raw/                        ← data lake (Google Drive)
│
├── operational/   11 file   58.654 baris   "Apa yang terjadi pada bisnis"
│                  pelanggan, produk, toko, promosi, pesanan
│
├── events/         5 file   64.213 baris   "Apa yang sedang terjadi, detik demi detik"
│                  pembayaran, refund, retur, support, aktivitas web
│
├── inventory/      1 file   35.040 baris   "Berapa stok hari ini"
│                  snapshot harian per produk per toko
│
└── reference/      2 file      863 baris   "Referensi & biaya marketing"
                   kota, belanja campaign
                                ─────────
                            Total   158.770 baris
```

KELUARGA	JAWAB PERTANYAAN	KARAKTER DATA	BARIS
operational	Apa yang terjadi pada bisnis?	Sudah final, satu baris = satu entitas	58.654
events	Apa yang sedang terjadi sekarang?	Bertambah terus, sering terulang	64.213
inventory	Berapa stok hari ini?	Snapshot harian, bisa hilang	35.040
reference	Berapa biaya marketing & di mana lokasinya?	Kecil, sebagai pembanding	863
Total			**158.770**

PERBEDAAN MENDASAR: TABEL TRANSAKSI VS TABEL EVENT

Dua kelompok ini sering tertukar, padahal cara memakainya sangat berbeda:

	TABEL TRANSAKSI (OPERATIONAL)	TABEL EVENT (EVENTS)
Satu baris =	Satu entitas final (satu order, satu produk)	Satu kejadian (satu "order diterima", satu "retur ditutup")
Bisa duplikat?	Harusnya tidak — kalau ada, itu bug	Wajar, satu order punya banyak event
Berisi status?	Ya, status akhir	Tidak — status justru harus disusun ulang dari urutan event
Diolah di	Silver sebagai dimensi/fakta	Silver sebagai log, lalu dirangkum di Gold

> **ATURAN EMAS MEMBACA DOKUMEN INI**
>
> Setiap tabel punya empat bagian — **Apa ini** (definisi), **Satu baris =** (grain), **Kolom** (arti tiap kolom), dan **Tujuan** (ke tabel mana isinya mengalir). Bagian **Masalah** hanya muncul kalau tabel itu memang sengaja dibuat bermasalah oleh instructor.

A · operational/

11 file · 58.654 baris · inti bisnis

Folder terbesar dan paling penting. Semua angka revenue di Gold pada akhirnya berakar dari dua tabel di sini: orders dan order_items.

`customers.json` 2.500 baris

APA INI

Daftar master seluruh pelanggan perusahaan. Satu identitas permanen, tidak berubah-ubah.

SATU BARIS =

Satu pelanggan (kunci customer_id)

KOLOM	TIPE	ARTINYA
`customer_id`	text	Kunci utama. Nilai seperti CUST-00001
`city_id`	text	Kota domisili. Menunjuk ke city_reference
`customer_segment`	text	Kelas pelanggan. Tiga nilai: consumer (813), enterprise (895), small_business (792)
`created_at_utc`	timestamptz	Tanggal pelanggan pertama kali terdaftar
`source_row_id`	text	Nomor baris di file sumber — dipakai untuk melacak asal-usul


```
{"customer_id": "CUST-00001", "city_id": "JKT", "customer_segment": "consumer",
 "created_at": "2025-07-29", "source_row_id": "CUSTOMER-ROW-000001"}
```


TUJUAN

Menjadi dimensi silver.customers, lalu jadi acuan gold.customer_daily dan gold.executive_kpis_daily (menghitung pelanggan aktif per hari).

BEDA DENGAN CUSTOMER_PROFILES

Tabel ini hanya menyimpan versi terbaru saja. Kalau ingin melihat riwayat perubahan segmen/kota, pakai customer_profiles.json.

`customer_profiles.json` 2.777 baris

APA INI

Riwayat versi profil pelanggan. Menyimpan setiap kali segmen atau kota pelanggan berubah, lengkap dengan rentang waktu berlakunya.

SATU BARIS =

Satu versi profil pada satu periode (kunci customer_id + valid_from_utc)

KOLOM	TIPE	ARTINYA
`customer_id`	text	Pelanggan yang dimaksud
`valid_from_utc`	timestamptz	Versi ini mulai berlaku sejak kapan
`valid_to_utc`	timestamptz	Versi ini berlaku sampai kapan
`city_id`	text	Kota selama periode ini
`customer_segment`	text	Segmen selama periode ini
`source_row_id`	text	Penanda baris sumber


```
{"customer_id": "CUST-00001", "city_id": "JKT", "customer_segment": "consumer",
 "valid_from_utc": "2025-07-29T00:00:00Z", "valid_to_utc": "2026-09-21T00:00:00Z"}
```


TUJUAN

Pola ini namanya **SCD Type-2** (Slowly Changing Dimension). Tanpa tabel ini, laporan "pelanggan enterprise menghasilkan berapa" akan selalu memakai segmen terbaru untuk semua transaksi lama — termasuk transaksi yang terjadi saat pelanggan itu masih consumer.

BUKTI DI DATA — MASALAH #7

2.777 baris untuk 2.500 pelanggan, artinya **277 pelanggan pernah berubah profil**. Bedanya bukan kebetulan — itu memang sengaja disisipkan.

`customer_addresses.json` 2.500 baris

APA INI

Daftar alamat pelanggan, juga disimpan dalam bentuk riwayat berlaku (SCD).

SATU BARIS =

Satu alamat pada satu periode berlaku (kunci address_id)

KOLOM	TIPE	ARTINYA
`address_id`	text	Kunci alamat, seperti ADDR-000001
`customer_id`	text	Pemilik alamat
`city_id`	text	Kota alamat tersebut
`valid_from_utc`	timestamptz	Mulai berlaku
`valid_to_utc`	timestamptz	Berakhir

TUJUAN

Menghubungkan pelanggan ke kota. Strukturnya sengaja disiapkan untuk SCD, tapi pada dataset ini tiap pelanggan baru punya satu baris (2.500 baris untuk 2.500 pelanggan, tidak ada yang ganda).

`products.json` 80 baris

APA INI

Master produk — daftar semua barang yang dijual beserta harga dasarnya.

SATU BARIS =

Satu produk (kunci product_id)

KOLOM	TIPE	ARTINYA
`product_id`	text	Kunci produk, seperti PROD-00001
`product_name`	text	Nama produk
`sku`	text	Kode unit penyimpanan gudang (barcode internal)
`unit_price`	numeric	Harga jual satuan, mis. 92.83


```
{"product_id": "PROD-00001", "product_name": "Synthetic Product 0001",
 "sku": "SKU-000001", "unit_price": 92.83}
```


TUJUAN

Menjadi silver.products. Setiap order_item harus menunjuk ke produk di sini — kalau tidak, item itu ditolak dengan alasan BROKEN_PRODUCT_REFERENCE.

MASALAH #5 YANG DISISIPKAN — REFERENSI PUTUS

Di file order_items ada **3 baris** yang menunjuk product_id yang tidak ada di tabel ini. Itu sengaja, untuk menguji validasi referensi.

`product_categories.json` 86 baris

APA INI

Riwayat kategori tiap produk. Menyimpan setiap kali sebuah produk berpindah kategori.

SATU BARIS =

Satu produk pada satu kategori selama rentang waktu tertentu

KOLOM	TIPE	ARTINYA
`product_id`	text	Produk yang dimaksud
`valid_from_utc / valid_to_utc`	timestamptz	Rentang kategori ini berlaku
`category_id`	text	Kode kategori, mis. CAT-02
`category_name`	text	Nama kategori. Enam nilai: beauty, electronics, fashion, grocery, home, sports

TUJUAN

Supaya laporan gold.product_daily memakai kategori yang berlaku pada tanggal penjualan, bukan kategori terbarunya. Contoh nyata: PROD-00013 berpindah dari *electronics* (sampai 2026-08-10) ke *home* (mulai 2026-08-11).

MASALAH #7 YANG DISISIPKAN — KATEGORI BERUBAH

86 baris untuk 80 produk — **6 produk** (PROD-00013, 00026, 00039, 00052, 00065, 00078) punya dua kategori. Bedanya bukan duplikat, tapi riwayat.

stores.json 5 baris

APA INI

Daftar toko fisik milik perusahaan.

SATU BARIS =

Satu toko (kunci store_id)

KOLOM	TIPE	ARTINYA
store_id	text	STORE-01 … STORE-05
store_name	text	Nama toko
city_id	text	Kota tempat toko berada (BDG, SBY, MDN, DPS, MKS)
location_type	text	Jenis lokasi — kelima toko bernilai physical

TUJUAN

Melengkapi order channel STORE. 2.517 order berasal dari toko fisik dan punya store_id; 7.483 order lainnya NULL karena datang dari channel online.

sales_channels.json 4 baris

APA INI

Daftar channel penjualan — cara pelanggan membeli.

SATU BARIS =

Satu channel (kunci channel_id)

KOLOM	TIPE	ARTINYA
channel_id	text	WEB, MOBILE_APP, MARKETPLACE, STORE
channel_name	text	Nama tampilan (sama nilainya dengan ID)

TUJUAN

Jadi acuan pembagian penjualan per channel di gold.channel_campaign_daily. Keempat channel hampir seimbang: STORE 2.517 · WEB 2.507 · MOBILE_APP 2.495 · MARKETPLACE 2.482 order.

PERHATIAN

Campaign di campaign_spend.csv hanya mencakup tiga channel digital — tidak ada campaign untuk channel STORE.

promotions.json 12 baris

APA INI

Daftar program promosi beserta periode berlakunya dan besarnya diskon.

SATU BARIS =

Satu program promosi (kunci promotion_id)

KOLOM	TIPE	ARTINYA
promotion_id	text	PROMO-001 … PROMO-012
promotion_code	text	Kode yang diketik pelanggan, mis. SAVE01
promotion_type	text	percentage (6 promosi) atau fixed_amount (6 promosi)
discount_rate	numeric	Besar diskon, mis. 0.1 = 10%
start_date / end_date	date	Periode promosi aktif


```
{"promotion_id": "PROMO-001", "promotion_code": "SAVE01", "promotion_type": "percentage",
 "discount_rate": 0.1, "start_date": "2026-06-26", "end_date": "2026-07-21"}
```


TUJUAN

Menghitung promotion_count di gold.product_daily — berapa banyak promosi yang aktif pada tanggal sebuah order dibuat.

MASALAH #10 YANG DISISIPKAN (PALING BERBAHAYA) — FACT MULTIPLICATION

12 promosi aktif bersamaan bisa membuat satu order ikut dihitung 12 kali kalau langsung di-join. Itulah *fact multiplication*, dan Gold menghindarinya dengan menghitung promosi lewat subquery, bukan join.

`orders.json` 10.001 baris → 10.000 unik

APA INI

Tabel terpenting: seluruh pesanan pelanggan beserta status dan nilai kiriman.

SATU BARIS =

Satu order (kunci order_id)

KOLOM	TIPE	ARTINYA
order_id	text	Kunci order, seperti ORD-000001
customer_id	text	Pembeli
ordered_at_utc	timestamptz	Tanggal & jam order. Rentang data: 22 Jun – 19 Sep 2026
sales_channel	text	Dibeli lewat apa (4 channel)
store_id	text / null	Toko fisik — NULL untuk order online (7.483 baris)
status	text	PLACED 3.492 · FULFILLED 2.561 · CONFIRMED 1.967 · CANCELLED 1.215 · RETURNED 765
shipping_revenue	numeric	Pendapatan ongkos kirim
updated_at_utc	timestamptz	Update terakhir — dipakai untuk memilih versi saat duplikat
source_row_id	text	Penanda baris sumber


```
{"order_id": "ORD-000001", "customer_id": "CUST-01757",
 "ordered_at_utc": "2026-08-20T06:16:00Z", "sales_channel": "MOBILE_APP",
 "shipping_revenue": 12.0, "status": "PLACED", "store_id": null,
 "updated_at_utc": "2026-08-20T23:16:00Z"}
```


TUJUAN

Menjadi silver.orders, lalu jadi tulang punggung seluruh lima tabel Gold. Semua revenue, AOV, dan KPI harian dihitung dari sini.

MASALAH #1 YANG DISISIPKAN — DUPLIKAT ORDER

File berisi 10.001 baris untuk 10.000 order unik. ORD-000001 muncul dua kali. Penanganan: ambil baris dengan updated_at_utc terbesar (versi terbaru), dan tandai baris pemenang dengan had_duplicate = true supaya duplikatnya tetap bisa diaudit. Terverifikasi: tepat **1 baris** bertanda duplikat di silver.orders.

`order_items.json` 24.960 baris

APA INI

Rincian isi tiap order — produk mana, berapa banyak, berapa harganya. Satu order bisa berisi banyak baris.

SATU BARIS =

Satu produk di dalam satu order (kunci order_item_id)

KOLOM	TIPE	ARTINYA
`order_item_id`	text	Kunci baris, seperti ITEM-000001-01
`order_id`	text	Order pemilik — harus ada di orders
`product_id`	text	Produk yang dibeli — harus ada di products
`quantity`	integer	Jumlah unit. Wajib > 0
`unit_price`	numeric	Harga satuan saat transaksi (bukan harga terbaru)
`item_discount_amount`	numeric	Diskon khusus baris ini
`source_row_id`	text	Penanda baris sumber


```
{"order_item_id": "ITEM-000001-01", "order_id": "ORD-000001", "product_id": "PROD-00012",
 "quantity": 3, "unit_price": 353.75, "item_discount_amount": 0}
```


TUJUAN

Sumber utama perhitungan uang di Gold. gross_merchandise_value = quantity × unit_price per order, dan dari sini juga dihitung gold.product_daily. Tiga dari empat alasan penolakan di dataset ini berasal dari tabel ini.

MASALAH #5 DAN #6 YANG DISISIPKAN — 4 BARIS TIDAK VALID

- **3 baris** menunjuk produk yang tidak ada → ditolak, kode BROKEN_PRODUCT_REFERENCE
- **1 baris** punya quantity ≤ 0 → ditolak, kode NEGATIVE_OR_ZERO_QUANTITY

Hasilnya: 24.960 → **24.956 baris valid**. Keempatnya tercatat di silver.rejected_records, tidak dihapus diam-diam.

order_promotions.json 15.729 baris KINI TERSEDIA

APA INI

Tabel jembatan yang menghubungkan order dengan promosi yang benar-benar dipakainya. Satu order bisa memakai lebih dari satu promosi, jadi satu order bisa muncul di beberapa baris.

SATU BARIS =

Satu pasang order + promosi (kunci order_id + promotion_id)

KOLOM	TIPE	ARTINYA
order_id	text	Order yang memakai promosi
promotion_id	text	Promosi yang dipakai — menunjuk ke promotions
discount_amount	numeric	Diskon yang benar-benar diterapkan pada pasangan ini
source_row_id	text	Penanda baris sumber, mis. ORDER-PROMO-ROW-0000001-PROMO-006


```
{"discount_amount": 5, "order_id": "ORD-000001", "promotion_id": "PROMO-006",
 "source_row_id": "ORDER-PROMO-ROW-0000001-PROMO-006"}
```


TUJUAN

Menghitung promotion_count yang akurat di gold.order_360 — persis berapa promosi yang dipakai tiap order. Terverifikasi: **9.064 order** memakai promosi, dengan **12 promosi** berbeda.

CATATAN STRUKTUR — FILE SUMBER BERBEDA DARI DUGAAN

File ini memakai nama kolom discount_amount, **bukan** applied_discount. Dan ia **tidak menyimpan** promotion_code — kode promosi diambil dengan mencarinya ke tabel silver.promotions berdasarkan promotion_id. Kedua penyesuaian ini didokumentasikan terbuka di build_silver.sql.

MASALAH #10 YANG DISISIPKAN — BERPOTENSI MENGGANDAKAN FACT

15.729 baris untuk 9.064 order berarti rata-rata satu order memakai **1,7 promosi**. Kalau tabel ini langsung di-join ke level order, satu order akan berlipat menjadi beberapa baris dan seluruh nilai revenue ikut tergandakan. Gold menghindarinya dengan mengagregasi COUNT(*) per order_id lebih dulu di dalam CTE, baru di-join.

RINGKASAN FOLDER OPERATIONAL/

11 file · 58.654 baris. Sepuluh di antaranya adalah data transaksi murni; satu (order_promotions) adalah tabel jembatan many-to-many antara order dan promosi.

B · events/

5 dari 5 file tersedia · 64.213 baris

Semua file event berbagi struktur yang sama: pembungkus event_id + event_type + dua timestamp, lalu data aslinya tersembunyi di dalam objek payload. Struktur berlapis ini meniru format log nyata.

```
{
  "event_id":         "EVT-00036639",          ← kunci unik (tapi bisa terulang = duplikat)
  "event_type":       "RETURN_CLOSED",          ← jenis kejadian
  "occurred_at_utc":  "2026-08-22T01:10:00Z",  ← KAPAN kejadiannya  ← dipakai untuk analitik
  "ingested_at_utc":  "2026-08-22T09:10:00Z",  ← KAPAN diterima sistem (bisa jauh lebih lambat)
  "payload": { ...data inti di sini... }         ← nested, bukan di root
}
```

DUA TIMESTAMP, DUA MASALAH

Kalau selisih ingested_at_utc − occurred_at_utc lebih dari 24 jam, event ditandai is_late = true. Dari seluruh data, **3.659 event terlambat**:

FILE EVENT	TOTAL	TERLAMBAT	%
web_events.json	30.000	1.807	6,0%
payment_events.json	21.915	1.254	5,7%
refund_events.json	3.982	240	6,0%
return_events.json	3.440	192	5,6%
support_events.json	2.870	166	5,8%
Total	**62.207**	**3.659**	**5,9%**

Event tetap diproses — hanya ditandai, karena membuangnya berarti menghilangkan transaksi sungguhan. Polanya konsisten: yang normal tiba dalam hitungan jam, yang terlambat tiba puluhan jam kemudian.

MASALAH #3 — EVENT TERLAMBAT DAN TIMESTAMP TIDAK BERURUTAN

Dua gejala sekaligus. Pertama, 3.659 event tiba lebih dari 24 jam setelah kejadiannya. Kedua, **urutan kedatangan event tidak kronologis** — retur bisa tercatat *received* sebelum *requested*. Karena itu status tidak diambil dari "event terakhir", melainkan dari **tahap tertinggi** yang pernah dicapai (requested → received → closed).

MASALAH #4 — TIMESTAMP DENGAN TIMEZONE BERBEDA

Tidak semua occurred_at_utc memakai akhiran Z (UTC). Sebanyak **5.837 event** tercatat dengan offset +07:00 (WIB) — sisa dari sistem sumber yang memakai waktu lokal Jakarta. Penanganan: semua timestamp di-cast ke timestamptz di Silver, sehingga PostgreSQL menormalkannya ke UTC secara otomatis. Tanggal bisnis juga dihitung dengan AT TIME ZONE 'UTC' secara eksplisit, jadi tidak bergantung setelan server.

`return_events.json` 3.553 baris → 3.440 unik

APA INI

Log proses retur barang. Satu retur menghasilkan beberapa event sepanjang tahapannya.

SATU BARIS =

Satu kejadian dalam proses retur (kunci event_id)

KOLOM	TIPE	ARTINYA
`event_id`	text	Kunci unik event
`event_type`	text	RETURN_REQUESTED · RETURN_RECEIVED · RETURN_CLOSED
`payload.order_id`	text	Order yang diretur
`payload.return_id`	text	ID retur, seperti RETURN-ORD-005899
`occurred_at_utc`	timestamptz	Waktu kejadian — ini yang dipakai untuk analitik
`ingested_at_utc`	timestamptz	Waktu diterima sistem (bisa selisih 1–56 jam)


```
{"event_id": "EVT-00036639", "event_type": "RETURN_CLOSED",
 "occurred_at_utc": "2026-08-22T01:10:00Z", "ingested_at_utc": "2026-08-22T09:10:00Z",
 "payload": {"order_id": "ORD-005899", "return_id": "RETURN-ORD-005899"}}
```


TUJUAN

Membangun kolom return_status di gold.order_360. Karena urutan event bisa tidak berurutan, status diambil dari tahap tertinggi yang pernah dicapai (requested → received → closed), bukan event terakhir yang datang. Juga mengisi return_count di gold.customer_daily.

MASALAH #1 YANG DISISIPKAN — EVENT DUPLIKAT

3.553 baris berisi hanya 3.440 event_id unik — **113 event terkirim dua kali** (simulasi retry). Penanganan: DISTINCT ON (event_id) memilih yang ingested_at_utc-nya terbaru, dan seluruh duplikatnya dicatat di rejected_records.

HASIL TAHAPAN DI GOLD

requested 1.372 · received 1.121 · closed 947 event. Setelah dirangkum per order: none 8.628 · closed 947 · requested 251 · received 174.

`support_events.json` 2.971 baris → 2.870 unik

APA INI

Log kontak pelanggan ke tim customer support — mencakup pembukaan dan penutupan tiket.

SATU BARIS =

Satu kejadian pada satu tiket support (kunci event_id)

KOLOM	TIPE	ARTINYA
`event_id`	text	Kunci unik event
`payload.ticket_id`	text	ID tiket — ada 1.619 tiket unik
`payload.customer_id`	text	Pelanggan yang menghubungi
`payload.order_id`	text	Order terkait (kalau ada)
`payload.reason`	text	Alasan kontak: product 445 · payment 444 · refund 405 · delivery 372
`event_type`	text	SUPPORT_TICKET_CREATED · SUPPORT_TICKET_CLOSED
`occurred / ingested`	timestamptz	Sama seperti tabel event lain


```
{"event_id": "EVT-00003409", "event_type": "SUPPORT_TICKET_CREATED",
 "payload": {"ticket_id": "TICKET-ORD-000554", "customer_id": "CUST-00549",
             "order_id": "ORD-000554", "reason": "refund"}}
```


TUJUAN

Mengisi dua metrik — support_contacts di gold.customer_daily (berapa kali pelanggan ini menghubungi support pada hari itu) dan support_contact_rate di gold.executive_kpis_daily (berapa persen pelanggan aktif hari itu menghubungi support).

DUA HAL YANG PERLU DIPERHATIKAN

- **101 event duplikat** (2.971 → 2.870 unik), ditangani sama seperti retur.
- Kolom reason kosong di 1.305 baris — itu baris CLOSED, yang memang tidak menyimpan alasan. Jangan dihitung sebagai "data hilang": jumlahnya persis sama dengan jumlah event penutupan.

Hasil tahapan: created 1.619 · closed 1.251 event.

payment_events.json

22.582 baris → 21.915 unik | KINI TERSEDIA | KRITIS

APA INI

Log pembayaran — mencatat setiap kali dana berhasil (atau gagal) ditagih untuk sebuah order.

SATU BARIS =

Satu percobaan/keberhasilan pembayaran (kunci event_id)

KOLOM	TIPE	ARTINYA
event_id	text	Kunci unik event
event_type	text	PAYMENT_CAPTURED 11.030 · PAYMENT_AUTHORIZED 10.302 · PAYMENT_FAILED 1.250
payload.order_id	text	Order yang dibayar
payload.amount	numeric	Nilai pembayaran. Bisa terbelah jadi beberapa event (split payment)
payload.payment_id	text	ID pembayaran, seperti PAY-ORD-001121
occurred / ingested	timestamptz	Sama seperti tabel event lain


```
{"event_id": "EVT-00006883", "event_type": "PAYMENT_CAPTURED",
 "occurred_at_utc": "2026-07-19T18:14:00Z", "ingested_at_utc": "2026-07-20T01:14:00Z",
 "payload": {"amount": 2172.62, "order_id": "ORD-001121", "payment_id": "PAY-ORD-001121"}}
```


TUJUAN

Mengisi captured_payment_amount dan payment_status di gold.order_360, serta menjadi penyebut dari refund_rate di gold.executive_kpis_daily.

MASALAH #2 YANG DISISIPKAN — SPLIT PAYMENT

1.915 order punya lebih dari satu event CAPTURED untuk order yang sama — dana ditagih bertahap. Kalau hanya satu event yang diambil, nilai pembayaran order itu akan kurang dari seharusnya. Penanganan: SUM(amount) FILTER (WHERE status IN ('CAPTURED', 'SETTLED', 'PAID', 'SUCCESS')) — seluruh potongan dijumlahkan, tapi hanya yang benar-benar tertagih.

STATUS PEMBAYARAN DI GOLD

Setelah digabungkan ke level order: captured 7.039 · refunded 1.991 · not_captured 970. Total dana tertagih terverifikasi: **13.137.988,14**.

MASALAH #8 YANG DISISIPKAN — ORDER DIBATALKAN TAPI PAYMENT SUDAH CAPTURED

Ini state yang tidak seharusnya terjadi: barang batal, tapi uang sudah masuk. Kode penanganannya **sudah ada** — payment_status = 'refunded' menang atas 'captured', dan refund_rate memakai penyebut "order yang payment-nya captured", bukan "semua order".

Namun pada dataset ini kasusnya nol. Terverifikasi: dari 1.215 order cancelled, **tidak satu pun** punya captured_payment_amount > 0. Jadi masalah ini disiapkan, tapi datanya belum memicunya — dan belum ada quality check khusus untuknya. Ini satu-satunya dari sepuluh masalah yang belum punya bukti angka.

`refund_events.json` 4.121 baris → 3.982 unik KINI TERSEDIA KRITIS

APA INI

Log pengembalian dana kepada pelanggan — pasangan dari payment_events.

SATU BARIS =

Satu aksi refund (kunci event_id)

KOLOM	TIPE	ARTINYA
`event_id`	text	Kunci unik event
`event_type`	text	REFUND_COMPLETED 2.064 · REFUND_ISSUED 2.057
`payload.order_id`	text	Order yang direfund
`payload.amount`	numeric	Nilai yang dikembalikan. Bisa sebagian (partial refund)
`payload.refund_id`	text	ID refund, seperti REF-ORD-006441
`occurred / ingested`	timestamptz	Sama seperti tabel event lain


```
{"event_id": "EVT-00040083", "event_type": "REFUND_ISSUED",
 "occurred_at_utc": "2026-06-25T16:16:00Z", "ingested_at_utc": "2026-06-25T20:16:00Z",
 "payload": {"amount": 425.51, "order_id": "ORD-006441", "refund_id": "REF-ORD-006441"}}
```


TUJUAN

Mengisi refunded_amount di gold.order_360 — komponen yang mengurangi net_revenue — serta refunded_units di gold.product_daily.

MASALAH #2 YANG DISISIPKAN — PARTIAL REFUND

1.943 order direfund sebagian saja (nilainya lebih kecil dari total sebelum refund), dan hanya **48 order** direfund penuh. Karena data refund tidak menyediakan rincian per item, refunded_units di Gold diestimasi proporsional: quantity × LEAST(1.0, refunded_amount / pre_refund_total). Rasio dibatasi maksimum 1,0 agar unit yang direfund tidak mungkin melebihi unit yang terjual — dan estimasi ini didokumentasikan terbuka, bukan disembunyikan.

DAMPAK YANG SUDAH TERVERIFIKASI

Total refund terhitung: **1.549.240,21**. Rekonsiliasi refund_amount_vs_refund_events berjalan (tidak lagi *skipped*) dengan **variance 0**.

`web_events.json` 30.986 baris → 30.000 unik KINI TERSEDIA

APA INI

Log kunjungan pengunjung ke website/aplikasi — halaman dilihat, produk diklik, keranjang diisi.

SATU BARIS =

Satu aktivitas pengunjung dalam satu sesi (kunci event_id)

KOLOM	TIPE	ARTINYA
`event_id`	text	Kunci unik event
`event_type`	text	PAGE_VIEW 10.345 · PURCHASE 10.333 · ADD_TO_CART 10.308
`payload.session_id`	text	ID sesi kunjungan — dipakai menghitung sesi unik
`payload.customer_id`	text	Pengunjung (kalau sudah login)
`payload.channel`	text	STORE · WEB · MOBILE_APP · MARKETPLACE
`payload.campaign_id`	text	Campaign yang dikaitkan ke kunjungan itu (bisa NULL)
`payload.order_id`	text	Order yang jadi dari kunjungan itu (bisa NULL)


```
{"event_id": "EVT-00045845", "event_type": "ADD_TO_CART",
 "occurred_at_utc": "2026-07-11T07:00:00Z", "ingested_at_utc": "2026-07-11T10:00:00Z",
 "payload": {"campaign_id": null, "channel": "STORE", "customer_id": "CUST-00..."}}
```


TUJUAN

Satu-satunya sumber untuk conversion_rate di gold.channel_campaign_daily (rumus: order ÷ sesi unik). Terverifikasi: kolom conversion_rate kini terisi di **855 dari 855 baris** — tidak ada lagi NULL.

Sesi unik dan aktivitas per channel

CHANNEL	SESI UNIK	EVENT
STORE	2.517	7.551
WEB	2.507	7.521
MOBILE_APP	2.494	7.482
MARKETPLACE	2.482	7.446
Total	**10.000**	**30.000**

TEMUAN MENARIK

Channel STORE **ikut tercatat** di web_events (7.551 event, terbanyak). Awalnya mudah disangka file ini hanya berisi aktivitas digital — ternyata tidak. Karena itu conversion_rate terisi juga untuk baris campaign channel STORE, walaupun campaign_spend.csv tidak punya campaign untuk STORE.

MASALAH #1 YANG DISISIPKAN — EVENT DUPLIKAT TERBANYAK

30.986 baris berisi 30.000 event_id unik — **986 event duplikat**, terbanyak di antara kelima file event. Penanganannya sama: ambil yang ingested_at_utc-nya terbaru, sisanya dicatat di rejected_records.

C · inventory/

1 file · 35.040 baris

Satu-satunya file CSV di keluarga ini, dan baris terbanyak kedua setelah order_items. Nilainya disimpan sebagai teks dan baru di-cast ke tipe sebenarnya di Silver.

`inventory_snapshots.csv` 35.040 baris

APA INI

Cuplikan kondisi stok gudang/toko setiap hari — berapa unit tersedia dan berapa yang dipesan pelanggan tapi belum dikirim.

SATU BARIS =

Satu produk di satu lokasi pada satu tanggal (kunci product_id + location_id + snapshot_date)

KOLOM	TIPE	ARTINYA
`product_id`	text	Produk yang distok
`location_id`	text	Lokasi: STORE-01 … STORE-05 (lima toko)
`snapshot_date`	date	Tanggal pencatatan. Rentang: 22 Jun – 19 Sep 2026
`available_quantity`	integer	Stok yang siap dijual
`reserved_quantity`	integer	Stok yang sudah dipesan, belum keluar
`unit_cost`	numeric	Harga pokok per unit (modal, bukan harga jual)

```
product_id,location_id,snapshot_date,available_quantity,reserved_quantity,unit_cost
PROD-00001,STORE-01,2026-06-22,2,1,52.94
```

TUJUAN

Mengisi available_inventory, reserved_inventory, dan stockout_flag di gold.product_daily. stockout_flag = true artinya stok tersedia = 0 pada tanggal itu.

MASALAH #9 YANG DISISIPKAN — SNAPSHOT YANG HILANG

90 tanggal × 5 lokasi = **450 pasangan**, dan semuanya ada. Tapi kalau dihitung lengkap: 80 produk × 450 pasangan = 36.000 baris seharusnya, sedangkan isinya hanya 35.040 — jadi ada **960 baris yang hilang**.

Penanganannya: kalau produk tidak punya baris pada tanggal tertentu, stockout_flag diisi **NULL**, bukan true. "*Tidak ada data*" ≠ "*stok habis*". Kalau keduanya disamakan, stockout_rate di laporan akan terlihat jauh lebih tinggi dari kenyataan.

MASALAH #10 YANG DISISIPKAN — RISIKO PENGGANDAAN

Tabel ini punya baris per lokasi. Kalau langsung di-join ke level item order, satu unit penjualan bisa terhitung sebanyak 5 (jumlah lokasi). Gold menghindarinya dengan menjumlahkan stok per produk-hari dulu di dalam CTE, baru join.

D · reference/

2 file · 863 baris · pembanding & biaya

city_reference.json 8 baris

APA INI

Daftar kota — kamus singkat yang menerjemahkan kode kota menjadi nama dan negara.

SATU BARIS =

Satu kota (kunci city_id)

KOLOM	TIPE	ARTINYA
city_id	text	Kode kota: JKT, BDG, SBY, MDN, DPS, MKS, BPN, PLM
city_name	text	Nama kota
country	text	Negara — seluruhnya ID (Indonesia)

TUJUAN

Mengubah kode kota pada pelanggan dan toko menjadi nama yang bisa dibaca. Delapan kota ini dipakai bersama oleh silver.customers dan silver.stores.

campaign_spend.csv 855 baris

APA INI

Catatan berapa uang yang dibelanjakan perusahaan untuk iklan, per hari per campaign per channel.

SATU BARIS =

Satu campaign pada satu channel pada satu tanggal

KOLOM	TIPE	ARTINYA
spend_date	date	Tanggal belanja. Rentang: 22 Jun – 19 Sep 2026
campaign_id	text	Sepuluh campaign: CMP-01 … CMP-10
channel	text	WEB, MOBILE_APP, MARKETPLACE — tidak termasuk STORE
spend_amount	numeric	Uang yang dikeluarkan hari itu

```
spend_date,campaign_id,channel,spend_amount
2026-06-22,CMP-01,MOBILE_APP,104.82
```

TUJUAN

Penggaris utama di gold.channel_campaign_daily — menghasilkan **ROAS** (net_revenue ÷ campaign_spend), yaitu berapa rupiah pendapatan yang dihasilkan tiap 1 rupiah iklan. 855 baris spend × 855 baris Gold = tabel Gold yang grain-nya identik dengan sumbernya.

MASALAH #10 YANG DISISIPKAN — TANTANGAN ATRIBUSI

File ini tidak mengatakan order mana yang datang dari campaign mana. Kalau setiap order di-join ke semua campaign pada tanggal & channel yang sama, revenue akan berlipat ganda. Solusinya: **atribusi round-robin** — order dibagi bergantian ke campaign, sehingga setiap order jatuh tepat ke satu campaign. Totalnya tetap sama: **7.483 = 7.483** (variance 0).

2 · Ke mana setiap tabel mengalir

Bagian "Tujuan" di atas dirangkum dalam satu peta. Baca dari kiri ke kanan: file sumber → tabel Silver → tabel Gold yang memakainya.

FILE SUMBER	BARIS	TABEL SILVER	ALIRAN KE GOLD
`customers.json`	2.500	`silver.customers`	customer_daily, executive_kpis_daily
`customer_profiles.json`	2.777	`silver.customer_profiles`	— (siap, belum dipakai Gold)
`customer_addresses.json`	2.500	`silver.customer_addresses`	— (siap, belum dipakai Gold)
`products.json`	80	`silver.products`	product_daily
`product_categories.json`	86	`silver.product_categories`	product_daily (kategori sesuai tanggal)
`stores.json`	5	`silver.stores`	order_360 (store_id)
`sales_channels.json`	4	`silver.sales_channels`	channel_campaign_daily
`promotions.json`	12	`silver.promotions`	product_daily (promotion_count)
`orders.json`	10.001	`silver.orders`	semua 5 tabel Gold
`order_items.json`	24.960	`silver.order_items`	order_360, product_daily
`order_promotions.json`	15.729	`silver.order_promotions`	order_360 (promotion_count)
`payment_events.json`	22.582	`silver.payment_events`	order_360, executive_kpis_daily
`refund_events.json`	4.121	`silver.refund_events`	order_360, product_daily
`return_events.json`	3.553	`silver.return_events`	order_360 (return_status), customer_daily
`support_events.json`	2.971	`silver.support_events`	customer_daily, executive_kpis_daily
`web_events.json`	30.986	`silver.web_events`	channel_campaign_daily (conversion_rate)
`inventory_snapshots.csv`	35.040	`silver.inventory_snapshots`	product_daily (stok & stockout)
`campaign_spend.csv`	855	`silver.campaign_spend`	channel_campaign_daily (ROAS)
`city_reference.json`	8	`silver.city_reference`	— (referensi pelengkap)
(tabel buatan pipeline)	2.010	`silver.rejected_records`	— (bukti audit penolakan)

HASIL AKHIR TIAP LAYER

LAYER	TABEL	BARIS	KETERANGAN
Bronze	3	158.770	Payload mentah, belum diubah
Silver	20	158.769	Bertipe & tervalidasi (berkurang 1 karena dedup order)
Gold	5	27.710	Analitik siap pakai
Ops	4	—	Log pipeline, quality check, rekonsiliasi

3 · Sepuluh masalah yang sengaja disisipkan

Sesuai README bagian 1. Setiap masalah muncul di satu tabel dan diuji oleh satu pemeriksaan.

Ini peta untuk berdiskusi saat ditanya "*masalah apa yang ada di dataset ini?*". Urutannya mengikuti README.

#	MASALAH	ADA DI	PENANGANAN	BUKTI
1	Duplikat order	`orders.json`	ROW_NUMBER() ambil versi terbaru + tanda had_duplicate	1 baris
1	Duplikat event	`5 file events`	DISTINCT ON (event_id) + dicatat DUPLICATE_EVENT_DROPPED	2.006 baris
2	Split payment	`payment_events.json`	SUM(...) FILTER (WHERE status IN (...)) — jumlahkan potongannya	1.915 order
2	Partial refund	`refund_events.json`	Estimasi proporsional LEAST(1.0, refunded / pre_refund_total)	1.943 order
3	Event terlambat	`5 file events`	Tandai is_late = true — tetap dipakai, tidak dibuang	3.659 event
3	Urutan event tak kronologis	`return_events.json`	Ambil **tahap tertinggi** (MAX), bukan event terakhir	3 tahap
4	Timezone berbeda	`5 file events`	Cast ke timestamptz + tanggal bisnis pakai AT TIME ZONE 'UTC'	5.837 event
5	Referensi produk putus	`order_items.json`	Ditolak + dicatat BROKEN_PRODUCT_REFERENCE	3 baris
5	Referensi order putus	`order_items.json`	Ditolak + dicatat BROKEN_ORDER_REFERENCE	0 baris
6	Quantity negatif/nol	`order_items.json`	Ditolak + dicatat NEGATIVE_OR_ZERO_QUANTITY	1 baris
7	Profil pelanggan berubah	`customer_profiles.json`	SCD Type-2: valid_from / valid_to / is_current	277 versi
7	Kategori produk berubah	`product_categories.json`	Pilih kategori yang berlaku pada tanggal transaksi, bukan terbaru	6 versi
8	Order batal tapi payment captured	`orders + payment_events`	payment_status = 'refunded' menang atas 'captured'	0 kasus
9	Inventory snapshot hilang	`inventory_snapshots.csv`	stockout_flag = NULL (bukan TRUE) — "tak ada data" ≠ "habis"	960 baris
10	Promotion menggandakan fact	`promotions + order_promotions`	Subquery COUNT(*), bukan join langsung	12 promosi
10	Campaign menggandakan fact	`campaign_spend.csv`	Atribusi round-robin: satu order = satu campaign	7.483 = 7.483

SATU-SATUNYA YANG BELUM PUNYA BUKTI ANGKA — #8

Kode penanganannya lengkap, tapi pada dataset ini **tidak ada satu pun** order cancelled yang payment-nya sudah captured (0 dari 1.215). Jadi masalah ini disiapkan, tapi datanya belum memicunya — dan belum ada quality check khusus yang menargetkannya. Ini jujur perlu disebut kalau ditanya.

4 · Ringkasan angka terverifikasi

Semua nilai di bawah diambil langsung dari database pada run terakhir.

METRIK	NILAI	SUMBER
Total baris mentah	158.770	Bronze, 19 file
Total baris Silver	158.769	20 tabel
Total baris Gold	27.710	5 tabel kontrak
Baris ditolak	2.010	silver.rejected_records
Gross merchandise value	15.100.182,54	gold.order_360
Net revenue	13.558.674,83	gold.order_360
Dana tertagih	13.137.988,14	captured_payment_amount
Total refund	1.549.240,21	refunded_amount
Item terjual	24.956	SUM(item_count)
Unit terjual	62.379	SUM(unit_quantity)
Order teratribusi campaign	7.483	channel_campaign_daily
Event terlambat	3.659	is_late = true
Event duplikat dibuang	2.006	5 file events
Grain violation (Gold)	0	5 pemeriksaan grain
Rekonsiliasi variance	0	11 rekonsiliasi, semua cocok

CATATAN PENUTUP

Seluruh isi dokumen ini diambil langsung dari file dataset dan schema pipeline yang ada di proyek. Angka baris adalah angka terverifikasi dari run terakhir. Keempat file yang sebelumnya bertanda *BELUM TERSEDIA* kini sudah tersedia, dan pipeline sudah mengisinya otomatis tanpa perubahan kode — termasuk menghidupkan kembali conversion_rate, captured_payment_amount, refunded_amount, dan promotion_count yang sebelumnya memakai nilai cadangan.